

Output Tracing

Question 1: Form Data and String Functions

Filename: form1.html

```
<!DOCTYPE html>
<html>
<body>
<form method="post" action="action1.php">
  <input type="text" name="name">
  <br><br>
  <input type="number" name="age">
  <br><br>
  <input type="submit" value="Submit">
</form>
<br>
<?php echo $result; ?>
</body>
</html>
```

Filename: action1.php

```
<?php
$result = "";
if ($ SERVER["REQUEST METHOD"] == "POST") {
  $name = isset($ POST['name']) ? $ POST['name'] : "";
  $age = isset($ POST['age']) ? $ POST['age'] : "";

  if (empty($name) && empty($age)) {
    $result = "Session starts here";
  }
  elseif (empty($name) && $age != "" && $age < 18) {
    $string = "form validation";
    $result = strtoupper(substr($string, 0, 4));
  }
  elseif (empty($name) && $age != "" && $age >= 18) {
    $string = "client server";
    $parts = explode(" ", $string);
    $result = strrev($parts[1]);
  }
  elseif (!empty($name) && empty($age)) {
    $string = "database connection";
    $result = str replace("connection", "query", $string);
  }
  elseif (!empty($name) && $age >= 18) {
    $letters = str split("mysqli");
    $result = count($letters) . " chars";
  }
}
?>
```

Write down the output for each of the following events:

Event	Output
Both name and age field submitted empty	
Name field empty and age field value = 15 submitted	
Name field empty and age field value = 22 submitted	
Name field value = "Rahim" and age field empty submitted	
Name field value = "Rahim" and age field value = 20 submitted	

Question 2: Superglobals, Arrays and Request Methods

Filename: form2.html

```
<!DOCTYPE html>
<html>
<body>
<form method="post" action="action2.php">
  <input type="text" name="email">
  <br><br>
  <input type="number" name="qty">
  <br><br>
  <input type="submit" value="Submit">
</form>
<br>
<?php echo $result; ?>
</body>
</html>
```

Filename: action2.php

```
<?php
$result = "";
if ($ SERVER["REQUEST METHOD"] == "POST") {
  $email = isset($ POST['email']) ? $ POST['email'] : "";
  $qty   = isset($ POST['qty'])   ? $ POST['qty']   : "";

  if (empty($email) && empty($qty)) {
    $result = "Nothing received";
  }
  elseif (empty($email) && $qty !== "" && $qty < 5) {
    $string = "session cookie";
    $pos = strpos($string, "cookie");
    $result = substr($string, 0, $pos - 1);
  }
  elseif (empty($email) && $qty !== "" && $qty >= 5) {
    $marks = [40, 25, 35];
    $total = array_sum($marks);
    if ($total > 90) { $result = "Grade A"; }
    else { $result = "Grade B"; }
  }
  elseif (!empty($email) && empty($qty)) {
    $string = "get and post";
    $words = explode(" ", $string);
    $result = strtoupper($words[0]) . "-" . strtoupper($words[2]);
  }
  elseif (!empty($email) && $qty >= 5) {
    $methods = ["GET", "POST", "PUT"];
    if (in_array("POST", $methods)) {
      $result = implode("/", array_reverse($methods));
    }
  }
}
?>
```

Write down the output for each of the following events:

Event	Output
Both email and qty field submitted empty	
Email field empty and qty field value = 3 submitted	
Email field empty and qty field value = 8 submitted	
Email field value = "a@b.com" and qty field empty submitted	
Email field value = "a@b.com" and qty field value = 6 submitted	

Question 3: String Trimming, Substrings and Associative Arrays

Filename: form3.html

```
<!DOCTYPE html>
<html>
<body>
<form method="post" action="action3.php">
  <input type="text" name="code">
  <br><br>
  <input type="number" name="price">
  <br><br>
  <input type="submit" value="Submit">
</form>
<br>
<?php echo $result; ?>
</body>
</html>
```

Filename: action3.php

```
<?php
$result = "";
if ($ SERVER["REQUEST METHOD"] == "POST") {
  $code = isset($ POST['code']) ? $ POST['code'] : "";
  $price = isset($ POST['price']) ? $ POST['price'] : "";

  if (empty($code) && empty($price)) {
    $result = "No data posted";
  }
  elseif (empty($code) && $price != "" && $price < 100) {
    $string = " ajax request ";
    $result = ucfirst(trim($string));
  }
  elseif (empty($code) && $price != "" && $price >= 100) {
    $string = "web technology";
    $result = substr($string, -10);
  }
  elseif (!empty($code) && empty($price)) {
    $string = "XML and JSON";
    if (str_contains($string, "JSON")) {
      $result = strtolower(substr($string, 0, 3));
    }
  }
  elseif (!empty($code) && $price >= 100) {
    $data = ["id" => 1, "name" => "Sabbir", "city" => "Dhaka"];
    $keys = array_keys($data);
    if (count($keys) === 3) {
      $result = implode(", ", $keys);
    }
  }
}
?>
```

Write down the output for each of the following events:

Event	Output
Both code and price field submitted empty	
Code field empty and price field value = 60 submitted	
Code field empty and price field value = 250 submitted	
Code field value = "P101" and price field empty submitted	
Code field value = "P101" and price field value = 150 submitted	

Answer Key – (Output Tracing)

Answer Key – Question 1

Event	Output
Both name and age field submitted empty	Session starts here
Name field empty and age field value = 15 submitted	FORM
Name field empty and age field value = 22 submitted	revres
Name field value = "Rahim" and age field empty submitted	database query
Name field value = "Rahim" and age field value = 20 submitted	6 chars

Explanation of each branch:

1. Both fields empty → the first if condition is true → "Session starts here".
2. Name empty, age = 15 → $15 < 18$ → `substr("form validation", 0, 4)` takes 4 characters from index 0 → "form" → `strtoupper()` → "FORM".
3. Name empty, age = 22 → $22 \geq 18$ → `explode(" ", "client server")` gives ["client", "server"], so `$parts[1]` is "server" → `strrev()` reverses it → "revres".
4. Name filled, age empty → `str_replace()` swaps "connection" with "query" → "database query".
5. Name filled, age = 20 → `str_split("mysql")` produces 6 single-character elements → `count()` is 6 → the dot operator joins it with " chars" → "6 chars".

Answer Key – Question 2

Event	Output
Both email and qty field submitted empty	Nothing received
Email field empty and qty field value = 3 submitted	session
Email field empty and qty field value = 8 submitted	Grade A
Email field value = "a@b.com" and qty field empty submitted	GET-POST
Email field value = "a@b.com" and qty field value = 6 submitted	PUT/POST/GET

Explanation of each branch:

1. Both fields empty → the first if condition is true → "Nothing received".
2. Email empty, qty = 3 → $3 < 5$ → `strpos("session cookie", "cookie")` returns 8 → `substr($string, 0, 7)` takes the first 7 characters → "session".
3. Email empty, qty = 8 → $8 \geq 5$ → `array_sum([40, 25, 35])` is 100 → $100 > 90$ is true → "Grade A".
4. Email filled, qty empty → `explode(" ", "get and post")` gives ["get", "and", "post"] → `$words[0]` is "get" and `$words[2]` is "post" → both uppercased and joined by "-" → "GET-POST".
5. Email filled, qty = 6 → `in_array("POST", $methods)` is true → `array_reverse()` gives ["PUT", "POST", "GET"] → `implode()` joins them with "/" → "PUT/POST/GET".

Answer Key – Question 3

Event	Output
Both code and price field submitted empty	No data posted
Code field empty and price field value = 60 submitted	Ajax request
Code field empty and price field value = 250 submitted	technology
Code field value = "P101" and price field empty submitted	xml
Code field value = "P101" and price field value = 150 submitted	id, name, city

Explanation of each branch:

1. Both fields empty → the first if condition is true → "No data posted".
2. Code empty, price = 60 → $60 < 100$ → `trim()` removes the leading and trailing spaces giving "ajax request" → `ucfirst()` capitalises only the first letter → "Ajax request".
3. Code empty, price = 250 → $250 \geq 100$ → `substr("web technology", -10)` counts 10 characters back from the end → "technology".
4. Code filled, price empty → `str_contains()` finds "JSON" (`str_contains` is case sensitive, and the string holds "JSON" in capitals) → `substr($string, 0, 3)` gives "XML" → `strtolower()` → "xml".
5. Code filled, price = 150 → `array_keys()` returns ["id", "name", "city"] → `count()` is 3 so the condition is true → `implode(" ", $keys)` → "id, name, city".

Notes for the Instructor

1. All three scripts follow the same branching pattern as the original sample, so the questions can be used interchangeably or combined into one paper.
2. Watch the `empty()` trap: `empty("0")` returns true in PHP, so a submitted value of 0 falls into the first branch rather than the "less than" branch. Avoid 0 as a test value unless you intend to examine this behaviour.
3. As in the original sample, the echo statement sits in the .html file. If students run these files on a real server, rename the form file to .php (or move the echo into the action file) so the PHP tag is actually parsed.
4. Every output in this key was verified by executing the scripts on PHP 8.3.